

EXP NO:02	EXPLORATORY DATA ANALYSIS (EDA) – DATA IMPORT AND EXPORT
------------------	---

AIM:

To perform exploratory data analysis on the Baseball Databank dataset using Python and Pandas to understand its structure, characteristics, and statistical properties.

PROCEDURE:

1. Import the required Python libraries such as Pandas, NumPy, and Matplotlib.
2. Load the Baseball Databank dataset into a Pandas DataFrame.
3. Display the first and last few records of the dataset.
4. Examine the number of rows, columns, and data types.
5. Check for missing and duplicate values.
6. Calculate the descriptive statistics of numerical attributes.
7. Identify relationships and patterns among the important variables.
8. Create suitable visualizations to understand the distribution of the data.
9. Analyze the results and summarize the important observations.

OUTPUT:

```
In [1]: import pandas as pd
import sqlite3
```

```
In [2]: from pathlib import Path

def find_data(filename):
    candidates = [
        Path(filename),
        Path("data") / filename,
        Path("../data") / filename,
        Path("../..data") / filename,
        Path("/content") / filename,
    ]
    for p in candidates:
        if p.exists():
            return p
    raise FileNotFoundError(f"{filename} not found. Upload the Baseball Databank CSV or keep it in the repository data folder.")

batting = pd.read_csv(find_data("Batting.csv"))
print("Baseball CSV file imported successfully.")
print("Rows:", len(batting))
print("Columns:", len(batting.columns))
```

Baseball CSV file imported successfully.
Rows: 101332
Columns: 22

```
In [3]: display(batting.head())
```

	playerID	yearID	stint	teamID	lgID	G	AB	R	H	2B	3B	HR	RBI	SB	CS	BB	SO	IBB	HBP	SH	SF	GIDP
0	abercda01	1871	1	TRO	NaN	1	4.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	NaN	NaN	NaN	NaN	NaN
1	addybo01	1871	1	RC1	NaN	25	118.0	30.0	32.0	6.0	0.0	0.0	13.0	8.0	1.0	4.0	0.0	NaN	NaN	NaN	NaN	NaN
2	allisar01	1871	1	CL1	NaN	29	137.0	28.0	40.0	4.0	5.0	0.0	19.0	3.0	1.0	2.0	5.0	NaN	NaN	NaN	NaN	NaN
3	alliso01	1871	1	WS3	NaN	27	133.0	28.0	44.0	10.0	2.0	2.0	27.0	1.0	1.0	0.0	2.0	NaN	NaN	NaN	NaN	NaN
4	ansonca01	1871	1	RC1	NaN	25	120.0	29.0	39.0	11.0	3.0	0.0	16.0	6.0	2.0	2.0	1.0	NaN	NaN	NaN	NaN	NaN

```
In [4]: batting.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 101332 entries, 0 to 101331
Data columns (total 22 columns):
#   Column      Non-Null Count  Dtype
---  ---
0   playerID    101332 non-null  object
1   yearID      101332 non-null  int64
2   stint       101332 non-null  int64
3   teamID      101332 non-null  object
4   lgID        100595 non-null  object
5   G           101332 non-null  int64
6   AB          96183 non-null   float64
7   R           96183 non-null   float64
8   H           96183 non-null   float64
9   2B          96183 non-null   float64
10  3B          96183 non-null   float64
11  HR          96183 non-null   float64
12  RBI         95759 non-null   float64
13  SB          94883 non-null   float64
14  CS          72729 non-null   float64
15  BB          96183 non-null   float64
16  SO          88345 non-null   float64
17  IBB         59620 non-null   float64
18  HBP         93373 non-null   float64
19  SH          89845 non-null   float64
20  SF          60151 non-null   float64
21  GIDP        70075 non-null   float64
dtypes: float64(16), int64(3), object(3)
memory usage: 17.0+ MB
```

```
In [5]: display(batting.describe(include="all").T)
```

	count	unique	top	freq	mean	std	min	25%	50%	75%	max
playerID	101332	18659	mcguide01	31	NaN	NaN	NaN	NaN	NaN	NaN	NaN
yearID	101332.0	NaN	NaN	NaN	1963.506533	38.628278	1871.0	1933.0	1972.0	1997.0	2015.0
stint	101332.0	NaN	NaN	NaN	1.077567	0.283676	1.0	1.0	1.0	1.0	5.0
teamID	101332	149	CHN	4818	NaN	NaN	NaN	NaN	NaN	NaN	NaN
lgID	100595	6	NL	51385	NaN	NaN	NaN	NaN	NaN	NaN	NaN
G	101332.0	NaN	NaN	NaN	51.400111	47.145273	0.0	13.0	34.0	80.25	165.0
AB	96183.0	NaN	NaN	NaN	149.970327	186.557072	0.0	7.0	57.0	251.0	716.0
R	96183.0	NaN	NaN	NaN	19.887038	28.671365	0.0	0.0	5.0	30.0	192.0
H	96183.0	NaN	NaN	NaN	39.261647	53.310941	0.0	1.0	11.0	63.0	262.0
2B	96183.0	NaN	NaN	NaN	6.637067	9.801563	0.0	0.0	1.0	10.0	67.0
3B	96183.0	NaN	NaN	NaN	1.373361	2.710547	0.0	0.0	0.0	2.0	36.0
HR	96183.0	NaN	NaN	NaN	2.949305	6.409662	0.0	0.0	0.0	3.0	73.0
RBI	95759.0	NaN	NaN	NaN	17.965163	26.756514	0.0	0.0	4.0	27.0	191.0
SB	94883.0	NaN	NaN	NaN	3.158184	7.922994	0.0	0.0	0.0	2.0	138.0
CS	72729.0	NaN	NaN	NaN	1.324025	2.838196	0.0	0.0	0.0	1.0	42.0
BB	96183.0	NaN	NaN	NaN	13.811484	21.092775	0.0	0.0	3.0	20.0	232.0
SO	88345.0	NaN	NaN	NaN	21.629849	28.432978	0.0	2.0	10.0	30.0	223.0
IBB	59620.0	NaN	NaN	NaN	1.213234	2.894918	0.0	0.0	0.0	1.0	120.0
HBP	93373.0	NaN	NaN	NaN	1.113395	2.32066	0.0	0.0	0.0	1.0	51.0
SH	89845.0	NaN	NaN	NaN	2.4579	4.347818	0.0	0.0	1.0	3.0	67.0

SF	60151.0	NaN	NaN	NaN	1.150122	2.023981	0.0	0.0	0.0	2.0	19.0
GIDP	70075.0	NaN	NaN	NaN	3.210032	4.835881	0.0	0.0	1.0	5.0	36.0

```
In [6]: batting_sample = batting.head(20)
batting_sample.to_excel("baseball_sample.xlsx", index=False)

batting_excel = pd.read_excel("baseball_sample.xlsx")
print("Excel file imported successfully.")
display(batting_excel.head())
```

Excel file imported successfully.

	playerID	yearID	stint	teamID	lgID	G	AB	R	H	2B	3B	HR	RBI	SB	CS	BB	SO	IBB	HBP	SH	SF	GIDP
0	abercda01	1871	1	TRO	NaN	1	4	0	0	0	0	0	0	0	0	0	0	NaN	NaN	NaN	NaN	NaN
1	addybo01	1871	1	RC1	NaN	25	118	30	32	6	0	0	13	8	1	4	0	NaN	NaN	NaN	NaN	NaN
2	allisar01	1871	1	CL1	NaN	29	137	28	40	4	5	0	19	3	1	2	5	NaN	NaN	NaN	NaN	NaN
3	alliso01	1871	1	WS3	NaN	27	133	28	44	10	2	2	27	1	1	0	2	NaN	NaN	NaN	NaN	NaN
4	ansonca01	1871	1	RC1	NaN	25	120	29	39	11	3	0	16	6	2	2	1	NaN	NaN	NaN	NaN	NaN

```
In [7]: conn = sqlite3.connect("baseball_data.db")
batting_sample.to_sql("batting_records", conn, if_exists="replace", index=False)

query = "SELECT * FROM batting_records LIMIT 5"
batting_sql = pd.read_sql(query, conn)

print("SQL data imported successfully.")
display(batting_sql)
conn.close()
```

SQL data imported successfully.

```
In [8]: 1 print("Web import syntax demonstrated using pd.read_csv(url).")
2 print("Example: pd.read_csv(url) where url points to a CSV resource.")
```

Web import syntax demonstrated using pd.read_csv(url).
Example: pd.read_csv(url) where url points to a CSV resource.

```
1 batting_sample = batting.head(5)
3 print(batting_sample)
```

	playerID	yearID	stint	teamID	lgID	G	AB	R	H	2B	3B	HR	RBI	SB	CS	BB	SO	IBB	HBP	SH	SF	GIDP
0	abercda01	1871	1	TRO	NaN	1	4.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	NaN	NaN	NaN	NaN	NaN
1	addybo01	1871	1	RC1	NaN	25	118.0	30.0	32.0	6.0	0.0	0.0	13.0	1.0	4.0	NaN	0.0	NaN	NaN	NaN	NaN	NaN
2	allisar01	1871	1	CL1	NaN	29	137.0	28.0	40.0	4.0	5.0	0.0	19.0	3.0	1.0	2.0	5.0	NaN	NaN	NaN	NaN	NaN
3	alliso01	1871	1	WS3	NaN	27	133.0	28.0	44.0	10.0	2.0	2.0	27.0	1.0	1.0	0.0	2.0	NaN	NaN	NaN	NaN	NaN
4	ansonca01	1871	1	RC1	NaN	25	120.0	29.0	39.0	11.0	3.0	0.0	16.0	6.0	2.0	2.0	1.0	NaN	NaN	NaN	NaN	NaN

```
In [9]: 1 batting_sample.to_csv("baseball_export.csv", index=False)
2 print("Baseball CSV file exported successfully.")
3 print("Exported rows:", len(batting_sample))
```

Baseball CSV file exported successfully.
Exported rows: 20

```
In [10]: 1 batting_sample.to_excel("Baseball_Excel.xlsx", index=False)
2 print("Baseball Excel file exported successfully.")
3 print("Exported rows:", len(batting_sample))
```

Baseball Excel file exported successfully.
Exported rows: 20

```
In [11]: 1 batting_sample.to_json("baseball_export.json", orient="records", indent=2)
2 print("Baseball JSON file exported successfully.")
3 print("Exported records:", len(batting_sample))
```

Baseball JSON file exported successfully.
Exported records: 20

```
In [12]: 1 conn = sqlite3.connect("baseball_export.db")
2 batting_sample.to_sql("baseball_data", conn, if_exists="replace", index=False)
3
4 count = pd.read_sql("SELECT COUNT(*) AS row_count FROM baseball_data", conn)
5 print("Baseball data exported to SQLite successfully.")
6 display(count)
7 conn.close()
```

Baseball data exported to SQLite successfully.

	row_count
0	20

```
In [13]: 1 print("Dataset shape:", batting.shape)
2 print("Number of player-season records:", len(batting))
3 print("Seasons:", int(batting["yearID"].min()), "to", int(batting["yearID"].max()))
4 print("\nColumns:")
5 print(batting.columns.tolist())
```

Dataset shape: (101332, 22)
Number of player-season records: 101332
Seasons: 1871 to 2015

Columns:
['playerID', 'yearID', 'stint', 'teamID', 'lgID', 'G', 'AB', 'R', 'H', '2B', '3B', 'HR', 'RBI', 'SB', 'CS', 'BB', 'SO', 'IBB', 'HBP', 'SH', 'SF', 'GIDP']

RESULT:

The Baseball Databank dataset was successfully analyzed and visualized.